

PRACTICAL NO.1

```
import pandas as pd
df=pd.read_csv(r"C:\Users\sahil_exgi\Downloads\iris.csv")
df
df.describe()
df.isnull()
df.isnull().sum()
df['petal_length']
df['petal_length']=df['petal_length'].astype(int)
df['species'].replace({'setosa':0,'virginica':1,'versicular':2} inplace=True)
```

PRACTICAL NO.2

```
import pandas as pd
df=pd.read_csv(r"C:\Users\sahil_exgi\Downloads\Students_Performance - Students_Performance.csv")
df.head()
df.isnull().sum()
df
print(df.columns.tolist())
df.columns=df.columns.str.strip()
math_score_mean=df['Math_Score'].mean()
df['Math_Score']=df['Math_Score'].fillna(math_score_mean)
reading_score_mean=df['Reading_Score'].mean()
df['Reading_Score']=df['Reading_Score'].fillna(reading_score_mean)
writing_score_mean=df['Writing_Score'].mean()
df['Writing_Score']=df['Writing_Score'].fillna(writing_score_mean)
df
import matplotlib.pyplot as plt
df.boxplot(column=['Math_Score','Reading_Score','Writing_Score'])
plt.show()
```

PRACTICAL NO.4

Create a Linear Regression Model using Python to predict home prices using Boston Housing Dataset

https://docs.google.com/spreadsheets/d/1opDtID_AydVJDBIoJI38BmMskT_zGhx9MLsPRwl8_RU/edit?usp=sharing

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# 1. Load Data
df = pd.read_csv(r"C:\Users\sahil_exgi\Downloads\BostonHousing - BostonHousing.csv")
X = df.drop('medv', axis=1) # Features
y = df['medv']             # Target (Price)

# 2. Split Data (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 3. Train Model
model = LinearRegression()
model.fit(X_train, y_train)

# 4. Predict and Evaluate
predictions = model.predict(X_test)
print(f"MSE: {mean_squared_error(y_test, predictions):.2f}")
```

PRACTICAL NO.5

Implement logistic regression using Python to perform classification on Social Network Ads.csv dataset, Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset.

<https://www.kaggle.com/datasets/d4rklucif3r/social-network-ads>

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score


# 1. Load Data

df = pd.read_csv(r"C:\Users\sahil_exgi\Downloads\Social_Network_Ads.csv")
X = df[['Age', 'EstimatedSalary']]
y = df['Purchased']


# 2. Split Data (75% Train, 25% Test)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)


# 3. Feature Scaling (Crucial for Logistic Regression)

sc = StandardScaler()

X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)


# 4. Train Model

classifier = LogisticRegression()

classifier.fit(X_train, y_train)


# 5. Predict

y_pred = classifier.predict(X_test)
```

```
# 6. Compute Confusion Matrix and Metrics
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
tn, fp, fn, tp = cm.ravel()
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred)
```

```
recall = recall_score(y_test, y_pred)
```

```
error_rate = 1 - accuracy
```

```
# Output Results
```

```
print(f"TP: {tp}, FP: {fp}, TN: {tn}, FN: {fn}")
```

```
print(f"Accuracy: {accuracy:.2f}")
```

```
print(f"Error Rate: {error_rate:.2f}")
```

```
print(f"Precision: {precision:.2f}")
```

```
print(f"Recall: {recall:.2f}")
```

PRACTICAL NO.6

Implement Simple Naïve Bayes classification algorithm using Python on iris.csv dataset, Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset.

<https://www.kaggle.com/datasets/himanshunakrani/iris-dataset>

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.naive_base import GaussianNB

from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score


# 1. Load the Iris Dataset

# The Iris dataset contains 3 classes: Setosa, Versicolor, and Virginica
df = pd.read_csv("iris.csv")
X = df.drop('species', axis=1) # Features: Sepal/Petal length & width
y = df['species']           # Target: Species name


# 2. Split Data (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


# 3. Train Naive Bayes Model (Gaussian is used for continuous data)
classifier = GaussianNB()
classifier.fit(X_train, y_train)


# 4. Predict
y_pred = classifier.predict(X_test)


# 5. Compute Metrics
cm = confusion_matrix(y_test, y_pred)


# Since Iris has 3 classes, we calculate metrics using 'macro' averaging
```

```

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='macro')
recall = recall_score(y_test, y_pred, average='macro')
error_rate = 1 - accuracy

# For a multi-class Confusion Matrix, TP/FP/TN/FN are calculated per class
print("Confusion Matrix:\n", cm)
print(f"\nAccuracy: {accuracy:.2f}")
print(f"Error Rate: {error_rate:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")

```

How Naive Bayes Works

It calculates the probability of a sample belonging to a class based on the features. For example, "What is the probability this is a *Setosa* given its *Petal Length* is 1.4cm?"

Breaking Down the Output

1. **Confusion Matrix:** Since the Iris dataset has 3 species, your matrix will be 3×3 instead of 2×2 .
 - **Diagonal elements:** These are your **True Positives (TP)** for each specific class (where the model predicted correctly).
 - **Off-diagonal elements:** These represent the **Misclassifications** (FP and FN).
2. **Accuracy:** The percentage of flowers correctly identified out of the total.
3. **Precision:** Out of all flowers predicted as *Versicolor*, how many actually were?
4. **Recall:** Out of all actual *Versicolor* flowers in the dataset, how many did the model find?
5. **Error Rate:** The percentage of total mistakes made.

PRACTICAL NO.7

Extract Sample document and apply following document preprocessing methods: Tokenization, POS Tagging, stop words removal, Stemming and Lemmatization, Create representation of documents by calculating Term Frequency and Inverse Document Frequency.

```
import nltk

import pandas as pd

from nltk.tokenize import word_tokenize

from nltk.corpus import stopwords

from nltk.stem import PorterStemmer, WordNetLemmatizer

from sklearn.feature_extraction.text import TfidfVectorizer


# Download necessary NLTK data
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('averaged_perceptron_tagger_eng')
nltk.download('wordnet')


# 1. Sample Document
document = "The curious cats are catching small mice in the dark garden."


# 2. Tokenization (Breaking sentences into words)
tokens = word_tokenize(document.lower())


# 3. Stop Words Removal (Removing common words like 'the', 'is', 'in')
stop_words = set(stopwords.words('english'))

filtered_tokens = [w for w in tokens if w not in stop_words and w.isalnum()]


# 4. POS Tagging (Identifying Parts of Speech: Nouns, Verbs, etc.)
pos_tags = nltk.pos_tag(tokens)


# 5. Stemming (Chopping off ends of words)
```

```

stemmer = PorterStemmer()

stems = [stemmer.stem(w) for w in filtered_tokens]

# 6. Lemmatization (Reducing word to its dictionary root)

lemmatizer = WordNetLemmatizer()

lemmas = [lemmatizer.lemmatize(w, pos='v') for w in filtered_tokens]

# 7. TF-IDF Representation

corpus = [document, "Mice are small animals.", "The garden is dark."]

vectorizer = TfidfVectorizer()

tfidf_matrix = vectorizer.fit_transform(corpus)

# Display TF-IDF as a Table

df_tfidf = pd.DataFrame(tfidf_matrix.toarray(), columns=vectorizer.get_feature_names_out())

print("Original Tokens:", tokens)

print("POS Tags:", pos_tags[:5])

print("Lemmatized:", lemmas)

print("\nTF-IDF Matrix:\n", df_tfidf)

```

Method	Description	Example Output
Tokenization	Splitting text into individual units (tokens).	"cats" , "are" , "catching"
POS Tagging	Labeling words as Nouns (NN), Verbs (VB), etc.	('cats', 'NNS')
Stop Words	Deleting high-frequency words that carry little meaning.	Removes: <i>"the"</i> , <i>"is"</i> , <i>"in"</i>
Stemming	A "crude" way to find the root by cutting suffixes.	"catching" $\rightarrow$ "catch"
Lemmatization	A "smart" way to find the root using a dictionary.	"mice" $\rightarrow$ "mouse"

PRACTICAL NO.8

Use the inbuilt dataset 'titanic'. Use the Seaborn library to see if we can find any patterns in the data, Write a code to check how the price of the ticket (column name: 'fare') for each passenger is distributed by plotting a histogram.

```
import seaborn as sns

import matplotlib.pyplot as plt

# 1. Load the inbuilt Titanic dataset
titanic = sns.load_dataset('titanic')

# 2. Preview the data (optional)
# print(titanic.head())

# 3. Plot the distribution of 'fare'
plt.figure(figsize=(10, 6))
sns.histplot(titanic['fare'], bins=30, kde=True, color='teal')

# 4. Add labels and title
plt.title('Distribution of Passenger Fares on the Titanic')
plt.xlabel('Fare (Price of Ticket)')
plt.ylabel('Frequency')

# 5. Show the plot
plt.show()
```

Pattern Analysis: What are we looking for?

When you run this code, you are performing a **univariate analysis** (looking at one variable). Here is how to interpret the patterns in the histogram:

- **Skewness:** You will likely notice a "Right Skew" (positive skew). This means most passengers paid a very low fare (Third Class), while only a few paid extremely high prices (First Class).
- **Outliers:** The small bars far to the right represent the elite passengers who paid significantly more than the average.
- **KDE (Kernel Density Estimate):** The smooth line over the bars helps you see the "shape" of the data distribution, making it easier to identify the most common price points.

PRACTICAL NO.9

Use the inbuilt dataset 'titanic' as used in the above problem. Plot a box plot for distribution of age with respect to each gender along with the information about whether they survived or not. (Column names: 'sex' and 'age')

```
import seaborn as sns

import matplotlib.pyplot as plt

# 1. Load the dataset

titanic = sns.load_dataset('titanic')

# 2. Create the Box Plot

plt.figure(figsize=(10, 6))

sns.boxplot(data=titanic, x='sex', y='age', hue='survived')

# 3. Add details and titles

plt.title('Distribution of Age by Gender and Survival Status')

plt.xlabel('Gender')

plt.ylabel('Age')

plt.legend(title='Survived', loc='upper right', labels=['No', 'Yes'])

# 4. Display the plot

plt.show()
```

Feature Distributions (Histograms)

- **Petal Length & Width:** These features show a **bimodal distribution** (two peaks). There is a clear gap between one group of flowers (Setosa) and the other two. This makes these features excellent "predictors" for classification.
- **Sepal Length & Width:** These follow a more **normal distribution** (bell-shaped curve).

Comparison and Outliers (Boxplots)

- **Scale:** Petal measurements have a much wider range than sepal measurements.
- **Outliers:** In the **Sepal Width** feature, you will typically see a few dots above and below the whiskers. These represent outliers—flowers with unusually wide or narrow sepals compared to the rest of the group.
- **Separation:** You'll notice that the boxes for Petal Length do not overlap much across species, whereas Sepal measurements overlap significantly. This confirms that **Petal dimensions** are the most important features for distinguishing the species.

How to Interpret this Box Plot

This visualization allows you to see several patterns at once:

- **The "Box":** Represents the **Interquartile Range (IQR)**. The middle 50% of ages fall within this box.
- **The Median Line:** The line inside the box shows the median age for that group.
- **Whiskers:** The lines extending from the box show the range of the rest of the data.
- **Outliers (Dots):** Any dots above or below the whiskers represent unusual ages (like very elderly passengers) that fall outside the typical range.

PRACTICAL NO.10

Download the Iris flower dataset into a DataFrame, Scan the dataset and give the inference as: List down the features and their types (e.g., numeric, nominal) available in the dataset, Create a histogram for each feature in the dataset to illustrate the feature distributions, Create a boxplot for each feature in the dataset, Compare distributions and identify outliers.

```
import seaborn as sns

import matplotlib.pyplot as plt

import pandas as pd


# Load the dataset

iris = sns.load_dataset('iris')


# 1. Histograms (Feature Distributions)

iris.hist(edgecolor='black', linewidth=1.2, figsize=(12, 8), color='skyblue')

plt.suptitle("Histogram of Iris Features", fontsize=16)

plt.show()


# 2. Boxplots (Comparison & Outliers)

plt.figure(figsize=(12, 8))

sns.boxplot(data=iris, palette="Set2")

plt.title("Boxplot Comparison of Iris Features", fontsize=16)

plt.show()
```